



Discrepancy Learning for Dynamical System Identification

Dávid Gavallér
ES02RO

May 13, 2026

Abstract

This document presents an investigation into discrepancy learning methods for nonlinear dynamical systems using Sparse Identification of Nonlinear Dynamics (SINDy) and Neural Network based correction models. Several systems were examined, including a linear spring-mass-damper system, a falling ball with nonlinear aerodynamic drag, and the nonlinear Van der Pol oscillator. The systems were examined in MATLAB, where simulated measurement data was generated and used for system identification and discrepancy learning.

The objective of the work was to:

1. Reconstruct governing differential equations from simulated measurement data with SINDy to understand it
2. Investigate the effect of measurement noise and numerical differentiation
3. Apply sparse regression techniques to identify compact physical models
4. Introduce artificial discrepancies into physical systems
5. Learn and compensate these discrepancies using both SINDy and Neural Networks

The results demonstrate that SINDy can successfully recover interpretable governing equations, while discrepancy learning methods estimate and compensate for the difference between the nominal model and the observed system dynamics.

1 Introduction

Dynamic systems are traditionally modeled using first principles physics equations derived from fundamental laws. However, these models often fail to capture all the complexities of real-world systems because of numerous reasons:

- Simplifications
- Aging effects
- Unknown physical phenomenas

This leads to discrepancies between model predictions and observed behavior. Discrepancy learning methods aim to bridge this gap by learning the difference between the nominal model and the true system dynamics from data.

Data-driven system identification methods aim to reconstruct system dynamics directly from measured data. One important approach is Sparse Identification of Nonlinear Dynamics (SINDy) [1], which identifies governing equations using a predefined function library. The core assumption behind SINDy is that physical systems can be represented compactly using a set of candidate functions.

In addition to direct system identification, SINDy can be used for discrepancy learning by modeling the difference between a nominal model and observed dynamics alongside Neural Networks.

This project investigates the possibilities with SINDy for both system identification and discrepancy learning on several example systems, including a linear spring-mass-damper system, a falling ball with nonlinear drag, and the Van der Pol oscillator. Also, the results achieved with SINDy are compared to those obtained using Neural Networks for discrepancy learning.

2 Theoretical background

In this section, the theoretical background of the methods used in this project is reviewed.

2.1 Sparse Identification of Nonlinear Dynamics (SINDy) [1]

SINDy is a method for discovering the equation of a system from data. It assumes that the system can be described as a combination of a few candidate functions.

The time evolution of a dynamical system is generally expressed as:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)) \quad (1)$$

where $\mathbf{x}(t)$ is the state vector at time t and $\mathbf{f}$ is the function describing the system dynamics.

In order to determine $\mathbf{f}$, from data, measurements of $\mathbf{x}(t)$, and if possible, measurements of $\dot{\mathbf{x}}(t)$ are collected. The time derivatives can also be estimated from the state measurements using numerical differentiation techniques.

This gives the following two matrices:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^T(t_1) \\ \mathbf{x}^T(t_2) \\ \vdots \\ \mathbf{x}^T(t_m) \end{bmatrix} = \begin{bmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{bmatrix} \quad (2)$$

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}^T(t_1) \\ \dot{\mathbf{x}}^T(t_2) \\ \vdots \\ \dot{\mathbf{x}}^T(t_m) \end{bmatrix} = \begin{bmatrix} \dot{x}_1(t_1) & \dot{x}_2(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \dot{x}_2(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \dot{x}_2(t_m) & \cdots & \dot{x}_n(t_m) \end{bmatrix} \quad (3)$$

Next, a library of candidate functions $\Theta(\mathbf{X})$ is constructed, which includes various nonlinear functions of the state variables. For example, the library may include:

$$\Theta(\mathbf{X}) = [1 \quad \mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_n \quad \mathbf{x}_1^2 \quad \cdots \quad \mathbf{x}_n^2 \quad \mathbf{x}_1\mathbf{x}_2 \quad \cdots] \quad (4)$$

As it can be seen, there is a lot of freedom in the choice of the candidate functions, and it is important to include functions that are relevant to the system being studied based on prior knowledge.

To determine the active functions in the library, a regression problem is set up to determine the vectors of coefficients Ξ such that:

$$\dot{\mathbf{X}} \approx \Theta(\mathbf{X})\Xi \quad (5)$$

The least-squares solution to this problem is given by:

$$\Xi = (\Theta^T \Theta)^{-1} \Theta^T \dot{\mathbf{X}} \quad (6)$$

To promote sparsity in the solution, a thresholding step is applied to set small coefficients to zero, then the coefficient are recalculated using only the remaining active functions. This process is repeated until there is no change in the active functions.

3 Spring-Mass-Damper System

The first system investigated was a simple linear spring-mass-damper system, since it is a well-known system with a simple governing equation, easy to test SINDy on. The system consists of a mass m attached to a spring with spring constant k and a damper with damping constant c . The system is illustrated in Figure 1.

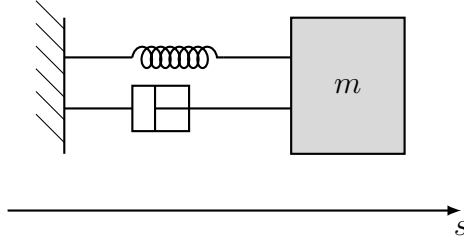


Figure 1: Mass-spring-damper system

The MATLAB implementation of the system started off from Newton's second law:

$$F = m \cdot a \quad (7)$$

Knowing that the forces acting on the mass are the spring force $F_s = -k \cdot s$ and the damping force $F_d = -c \cdot v$, the governing equation of the system can be written as:

$$-k \cdot s - c \cdot v = m \cdot a \quad (8)$$

This can be rearranged to express the acceleration:

$$a = -\frac{k}{m} \cdot s - \frac{c}{m} \cdot v \quad (9)$$

Next, by choosing specific values for the parameters m , k , and c , the ideal system was simulated in MATLAB to generate measurement data for s , as in a real scenario, it is most likely that only the position of the mass can be measured, the velocity and acceleration would have to be estimated from the position data.

The chosen parameters, initial conditions and simulation settings were:

- Mass $m = 1$ [kg]
- Spring constant $k = 4$ [N/m]
- Damping constant $c = 0.8$ [Ns/m]
- Initial position $s(1) = 1$ [m]
- Initial velocity $v(1) = 0$ [m/s]
- Simulation time $t = 0$ to 15 [seconds] with a time step of 0.01 [seconds]

The plot of the idealsimulated position, velocity and acceleration data is shown in Figure 2.

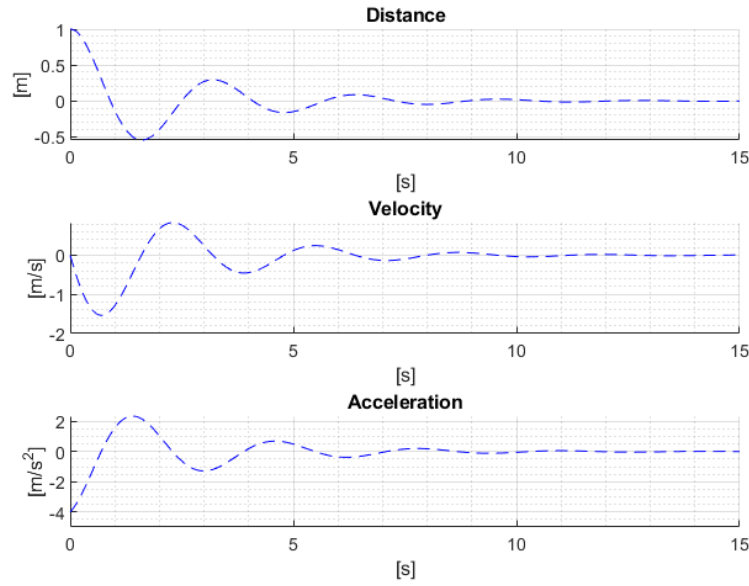


Figure 2: Simulated ideal position, velocity, and acceleration data for the spring-mass-damper system

Next, noise was added to the position data (gaussian with a noise level of 0.00001) to simulate measurement noise, the velocity and acceleration were estimated from the noisy position data using the `diff()` function, which calculates the difference between consecutive elements. The same principle was applied to the velocity data to estimate the acceleration. The results are shown in Figure 3.

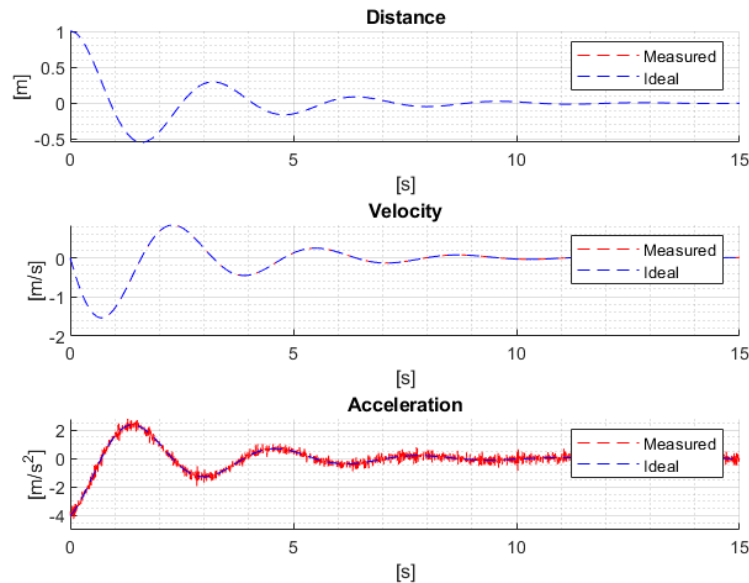


Figure 3: Simulated noisy position, velocity, and acceleration data for the spring-mass-damper system

As it can be seen, even this little noise becomes significant in the acceleration data, because of the numerical differentiation.

Next, it was time to test SINDy on the noisy "measurement" data. A library of candidate functions was constructed:

$$\Theta(\mathbf{X}) = [1 \quad v \quad s \quad vs \quad v^2 \quad s^2 \quad s^2v \quad sv^2 \quad s^3 \quad v^3] \quad (10)$$

The library was normalized with the RMS of each column for the Ξ coefficients to be comparable, and the regression was applied to determine the coefficients for each candidate function.

This way, the results for Ξ are the following (of course, the results may differ on each run because of the noise):

$$\Xi = \begin{bmatrix} 0.0001 \\ -0.3377 \\ -0.8990 \\ -0.0046 \\ -0.0007 \\ -0.0025 \\ -0.0048 \\ -0.0054 \\ -0.0046 \\ 0.0015 \end{bmatrix} \quad (11)$$

Since these coefficients are not sparse, a thresholding step was applied to set small coefficients to zero, then the coefficient were recalculated. Choosing a threshold of 0.01 and doing multiple iterations of this process, the final result was:

$$\Xi = \begin{bmatrix} 0 \\ -0.3354 \\ -0.8997 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (12)$$

These are not the final results, since the library was normalized, the coefficients need to be denormalized. After denormalization, the final results are:

$$\Xi = \begin{bmatrix} 0 \\ -0.8011 \\ -3.9972 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (13)$$

This means that the identified governing equation is:

$$a = -3.9972 \cdot s - 0.8011 \cdot v \quad (14)$$

This is very close to the true governing equation, which is:

$$a = -4 \cdot s - 0.8 \cdot v \quad (15)$$

As can be seen, the identification was successful and the coefficients are very close to the true values. Visual representation of the results is shown in Figure 4.

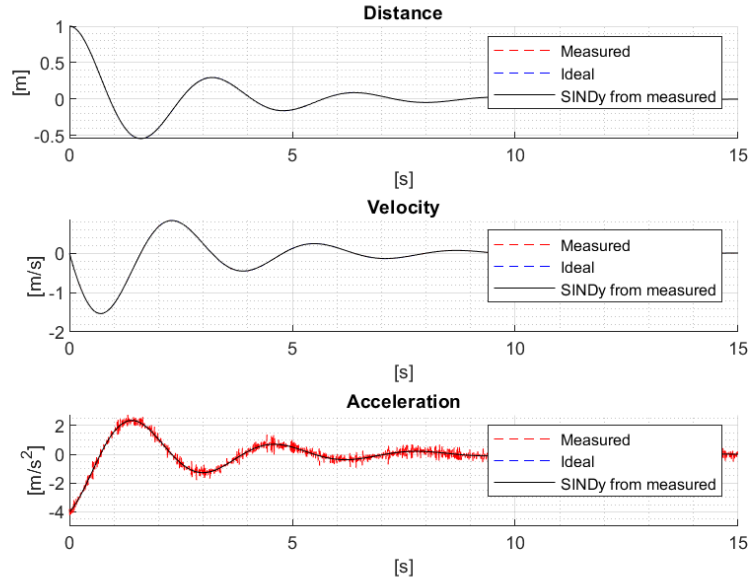


Figure 4: Identified governing equation for the spring-mass-damper system

The position and velocity values were calculated with forward Euler integration using the identified governing equation

After this, the same process was repeated with a higher noise level of 0.0001, which is 10 times higher than before. The results were not as good as before and the thresholding value had to be adjusted to achieve a sparse solution, but still the identification was successful and the identified governing equation was:

$$a = -4.0957 \cdot s - 0.9786 \cdot v \quad (16)$$

This is still close to the true governing equation, but the coefficients are not as accurate as before. Visual representation of the results is shown in Figure 5.

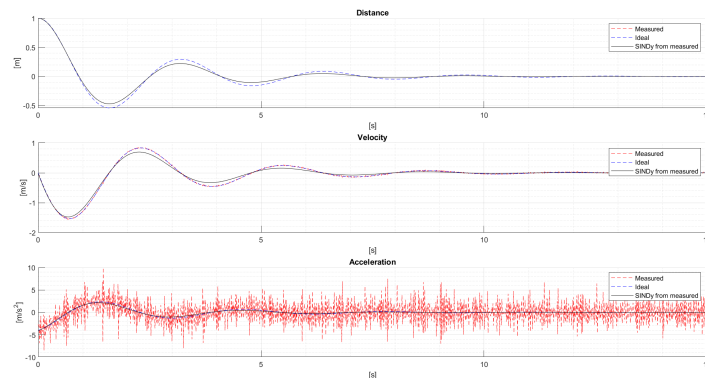


Figure 5: Identified governing equation for the spring-mass-damper system with higher noise level

By increasing the noise level even more to 0.001, the results completely got out of hand, by adjusting the thresholding value (to 4.5), a sparse solution could be achieved, but the identified governing equation was very far from the true one:

$$a = -31.2664 \cdot v + 13.373877 \cdot v^3 \quad (17)$$

Visual representation of the results is shown in Figure 6.

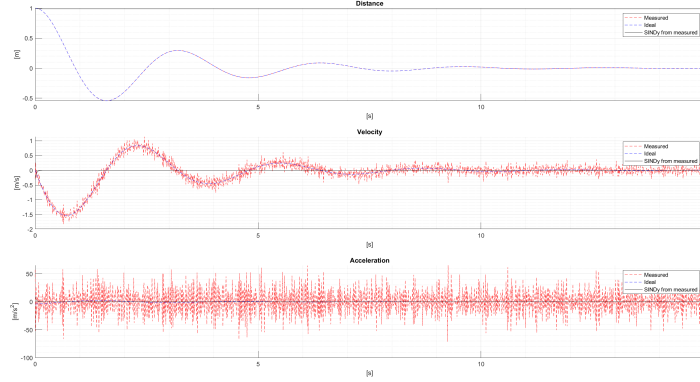


Figure 6: Identified governing equation for the spring-mass-damper system with very high noise level

To overcome this issue, instead of using the `diff()` function for numerical differentiation, a more robust method was used, first applying a Savitzky-Golay filter to the noisy position data to smooth it, then apply the `gradient()` function, because it calculates the numerical derivative using central differences, which is more accurate and less sensitive to noise than the `diff()` function. The parameters for the Savitzky-Golay filter were a window size of 21 and a polynomial order of 3. With only this much change, the results got much better. With the same 0.001 noise level, the identified governing equation was:

$$a = -4.022 \cdot s - 0.7619 \cdot v \quad (18)$$

Visual representation of the results is shown in Figure 7.

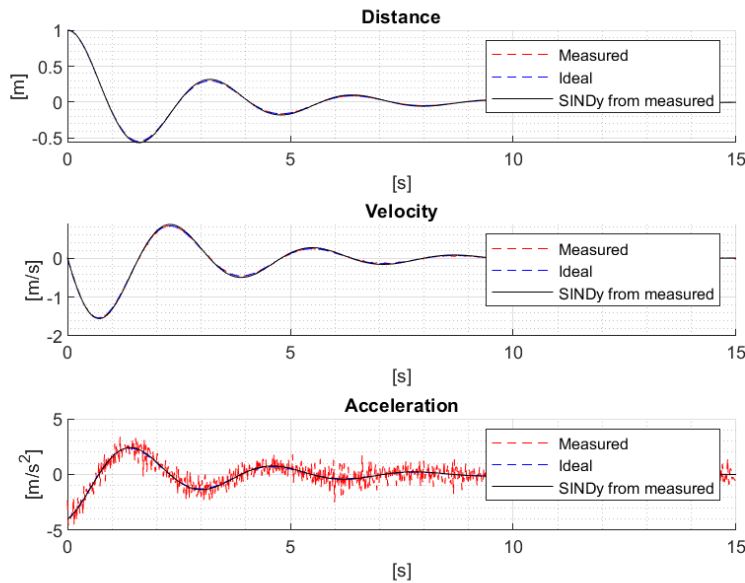


Figure 7: Identified governing equation for the spring-mass-damper system with smoothed data

Even with a noise level of 0.01, the following governing equation was identified:

$$a = -4.4315 \cdot s - 0.8010 \cdot v \quad (19)$$

Which can be considered very impressive by looking at the visualisation of the results in Figure 8.

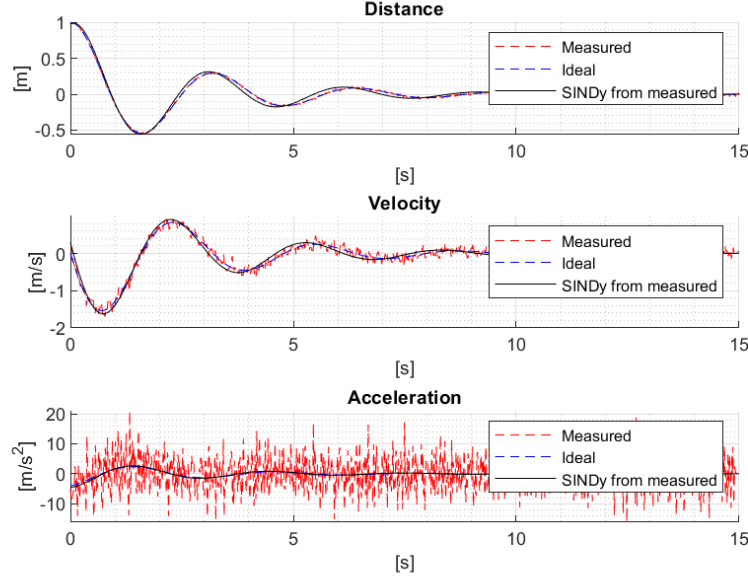


Figure 8: Identified governing equation for the spring-mass-damper system with smoothed data and very high noise level

Now that it is proven that smooting the data before numerical differentiation can significantly improve the results, making it more robust to noise, the next step was to test the effect of the choice of the candidate functions in the library. What happens if the functions in the library are linearly dependent? Intuition says that the coefficient weights should be distributed among the linearly dependent functions. Let's change $s \cdot v$ to s in the library, basially duplicating the s term, and see what happens. The noise level is reduced to 0, so that the effect of the linearly dependent functions can be observed more clearly, without the influence of noise. The library is now:

$$\Theta(\mathbf{X}) = [1 \quad v \quad s \quad s \quad v^2 \quad s^2 \quad s^2 v \quad s v^2 \quad s^3 \quad v^3] \quad (20)$$

After running the simulation, the coefficients couldn't be determined, MATLAB sent a warning about the matrix being singular. The inverse of a singular matrix does not exist, so the regression problem cannot be solved. This shows that the choice of the candidate functions in the library is very important, and linearly dependent functions should be avoided, otherwise the regression problem cannot be solved.

A way to avoid this happening is throwing out one of the linearly dependent functions. But of course, it is now always obvious which functions are linearly dependent. A way to determine the linearly dependent functions is through Singular Value Decomposition (SVD) of the library matrix Θ . The singular values of the matrix can be examined, and if there are any singular values that are close to zero, it indicates that there are linearly dependent functions in the library. By looking at the right singular vectors corresponding to the small singular values, the linearly dependent functions can be identified.

The singular values of the library matrix shown in equation (20) are:

$$\sigma = \begin{bmatrix} 90.6002 \\ 54.3295 \\ 40.3114 \\ 30.2025 \\ 24.7243 \\ 21.0753 \\ 12.7127 \\ 8.36104 \\ 5.06999 \\ 6.32417e - 15 \end{bmatrix} \quad (21)$$

As it can be seen, there is a singular value that is very close to zero, which indicates that there are linearly dependent functions in the library. Looking at the right singular vector corresponding to the small singular value:

$$\mathbf{v} = \begin{bmatrix} 1.4247e - 16 \\ -4.7779e - 17 \\ 0.7071 \\ -0.7071 \\ -1.9511e - 16 \\ -1.3038e - 16 \\ 1.7113e - 16 \\ 2.1797e - 16 \\ -5.8609e - 17 \\ 1.3930e - 16 \end{bmatrix} \quad (22)$$

By the third and fourth term being dominant and everything else being close to zero, it can be concluded that the third and fourth functions in the library are linearly dependent. After throwing out one of them and recalculating the coefficients, the following governing equation was identified:

$$a = -4 \cdot s - 0.8 \cdot v \quad (23)$$

Which is the true governing equation of the system. This shows that by using SVD, the linearly dependent functions in the library can be identified and thrown out, making the regression problem solvable and the correct governing equation identifiable.

Concluding the results for the spring-mass-damper system, it can be said that SINDy was able to successfully identify the governing equation of the system from noisy measurement data, and that smoothing the data before numerical differentiation can significantly improve the results. It was also shown that the choice of the candidate functions in the library is very important, and linearly dependent functions should be avoided, otherwise the regression problem cannot be solved. By using SVD, the linearly dependent functions can be identified and thrown out, making the regression problem solvable and the correct governing equation identifiable.

4 Ball Drop

The next system investigated was a falling ball with nonlinear aerodynamic drag. The ideal model is once again derived from Newton's second law:

$$F = m \cdot a \quad (24)$$

The forces acting on the ball is the gravitational force and the drag force:

$$-\frac{1}{2} \cdot \rho \cdot C_d \cdot A \cdot v^2 + m \cdot g = m \cdot a \quad (25)$$

When we learn basic physics, we usually interpret the drag coefficient C_d as a constant, but in reality it is not, it is a function of the Reynolds number, which is a function of the velocity. There are many levels of complexity to describe the drag force, but I chose the following equation in this project [2]:

$$C_d(Re) = \frac{24}{Re} + \frac{2.6 \frac{Re}{5.0}}{1 + \left(\frac{Re}{5.0}\right)^{1.52}} + \frac{0.411 \left(\frac{Re}{2.63 \cdot 10^5}\right)^{-7.94}}{1 + \left(\frac{Re}{2.63 \cdot 10^5}\right)^{-8.00}} + \frac{0.25 \left(\frac{Re}{10^6}\right)}{1 + \left(\frac{Re}{10^6}\right)} \quad (26)$$

Where the Reynolds number is calculated as:

$$Re(v) = \frac{\rho \cdot v \cdot d}{\mu} \quad (27)$$

Where ρ is the density of the medium, v is the velocity of the ball, d is the diameter of the ball, and μ is the dynamic viscosity of the fluid.

The characteristic of the drag coefficient is shown in Figure 9.

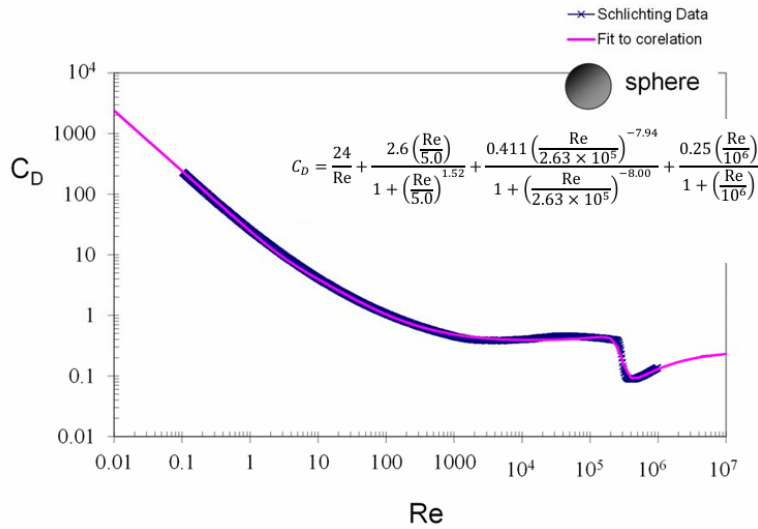


Figure 9: Drag coefficient as a function of the Reynolds number [2]

So the governing equation of the system looks like:

$$a = -\frac{1}{2} \cdot \rho \cdot C_d(Re(v)) \cdot A \cdot v^2 / m + g \quad (28)$$

The parameters, initial conditions and simulation settings for the system were:

- Mass $m = 0.045359$ [kg]
- Diameter $d = 0.021963 \cdot 2$ [m]
- Density of the medium $\rho = 1.225$ [kg/m³]

- Dynamic viscosity of the fluid $\mu = 1.81 \cdot 10^{-5}$ [kg/(m· s)]
- Initial position $s(1) = 0$ [m]
- Initial velocity $v(1) = 0$ [m/s]
- Simulation time $t = 0$ to 30 [seconds] with a time step of $\frac{1}{15}$ [seconds]
- Gravitational acceleration $g = 9.81$ [m/s²]

The acceleration, velocity and position of the ideal system visually can be seen in Figure 10.

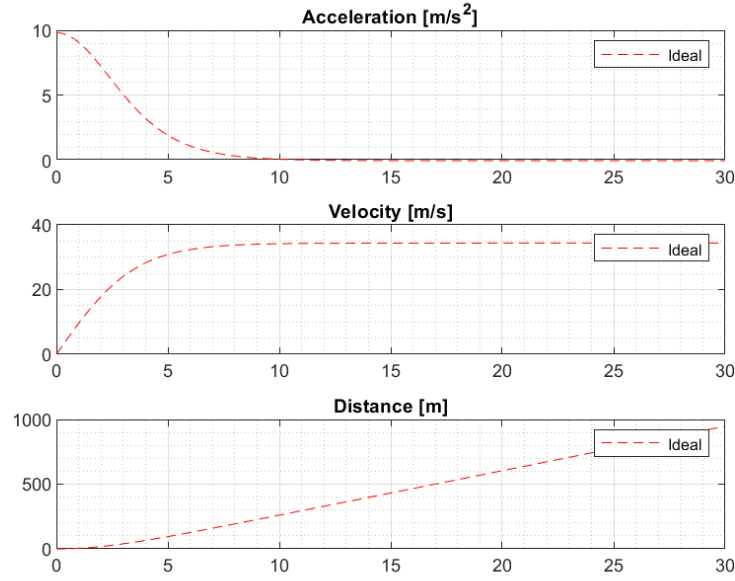


Figure 10: Simulated ideal position, velocity, and acceleration data for the falling ball system

The next step is to define a library of candidate functions for SINDy to identify the governing equation with. The library is the following:

$$\Theta = [1 \quad v \quad s \quad vs \quad v^2 \quad s^2 \quad s^2v \quad sv^2 \quad s^3 \quad v^3] \quad (29)$$

This library should be normalized with the RMS of each column for the coefficients to be indicators. After normalizing, as a lesson learned, the SVD of the library matrix should be examined to make sure there are no linearly dependent functions. The singular values of the library matrix are:

$$\sigma = \begin{bmatrix} 63.5443 \\ 20.6636 \\ 5.7834 \\ 3.2733 \\ 0.9206 \\ 0.3282 \\ 0.0953 \\ 0.0153 \\ 0.0009 \\ 0.0002 \end{bmatrix} \quad (30)$$

As it can be seen, there are several very small singular values, which is an indicator that there are linearly dependent functions of the library. The right singular vector corresponding to the smallest singular value is:

$$\mathbf{v} = \begin{bmatrix} 0.00006 \\ -0.00091 \\ 0.31734 \\ -0.47746 \\ -0.02456 \\ -0.56658 \\ 0.56936 \\ 0.15756 \\ -0.00104 \\ 0.02613 \end{bmatrix} \quad (31)$$

It can be seen that the third, fourth, sixth and seventh terms are dominant, which indicates that they are linearly dependent.

References

- [1] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. “Discovering governing equations from data by sparse identification of nonlinear dynamical systems”. In: *Proceedings of the National Academy of Sciences* 113.15 (Mar. 2016), pp. 3932–3937. ISSN: 1091-6490. DOI: 10.1073/pnas.1517384113. URL: <http://dx.doi.org/10.1073/pnas.1517384113>.
- [2] Faith A. Morrison. *Data Correlation for Drag Coefficient for Sphere*. Technical report, Department of Chemical Engineering, Michigan Technological University. 10 November 2016. Houghton, MI 49931, Nov. 2016.